WAWESCAPE

Tour Agency Web Application

Mini Project Report

Student	Index number
MBM Yoonus	EG/2023/5990

Project overview

WAWESCAPE is a full-stack Sri Lankan tour agency application. Customers browse and search tour packages, review itineraries, save favorites and request bookings. Administrators create, list, update and safely delete tour records stored persistently in MySQL.

The main assessed entity is the **tour**. The application demonstrates a complete form-based CRUD workflow, relational booking records, client and server validation, responsive navigation, reusable components and Git version history.

Repository: github.com/MuhammadYoonus/WAWESCAPE

Course: EC4307 Web Application Development. This report contains eight pages, including the cover.

1. Requirements and architecture

The application supports two roles. Customers can read published packages and submit authenticated booking requests; administrators manage tour packages and booking, inquiry and feedback statuses. Administrative API operations check the signed user's role.

Layer	Implementation and responsibility
Client	React, React Router and Tailwind CSS. Pages and reusable components provide forms, catalogue views, navigation, loading states and feedback.
HTTP API	Express/Node.js exposes REST endpoints. JWT middleware enforces authentication and admin authorization. Parameterized SQL queries access MySQL.
Persistence	MySQL stores users, tours, both booking types, inquiries and feedback. Startup migrations create required structures and preserve older records.
Optional notifications	SMTP and Twilio adapters can send confirmations when configured. Missing credentials skip those channels during local development.

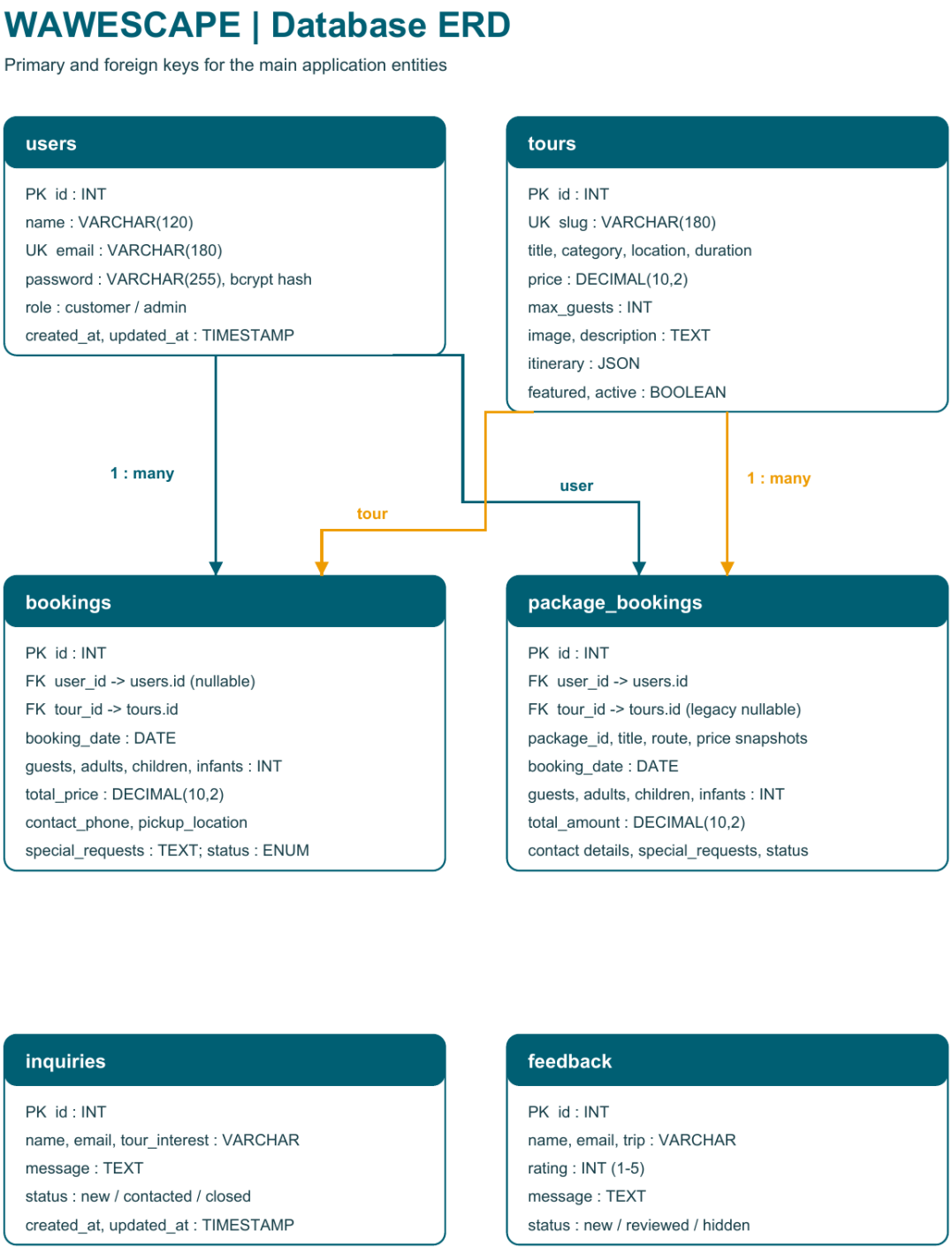
Main API operations

Operation	Endpoint	Behavior
Create	POST /api/tours	Admin form creates a validated tour.
Read	GET /api/tours	Lists public tours with search, filters and sorting.
Read details	GET /api/tours/:id	Returns a published tour and itinerary.
Update	PUT /api/tours/:id	Admin edits the existing tour record.
Delete	DELETE /api/tours/:id	Removes an unbooked tour; rejects booked tours.
Book	POST /api/package-bookings	Uses the stored tour price and capacity.

Flow: browser form -> authenticated HTTP request -> server validation -> parameterized SQL -> MySQL -> JSON response -> visible confirmation or error.

2. Database design

Primary/unique keys identify users and tours. Foreign keys connect bookings to the correct user and tour. Prices and totals use DECIMAL(10,2); dates use DATE and itineraries use JSON.



Package bookings retain price/title snapshots for history. Legacy trip IDs are migrated through explicit relationships, rather than assuming matching numeric IDs identify the same tour.

3. Create and Read

The admin form accepts a name, route, duration, price, guest limit, image, descriptions and itinerary. Saving inserts a tour in MySQL and provides a link to its public duration group. Public reads support name/location/description searches and approved price/name/newest sorting.

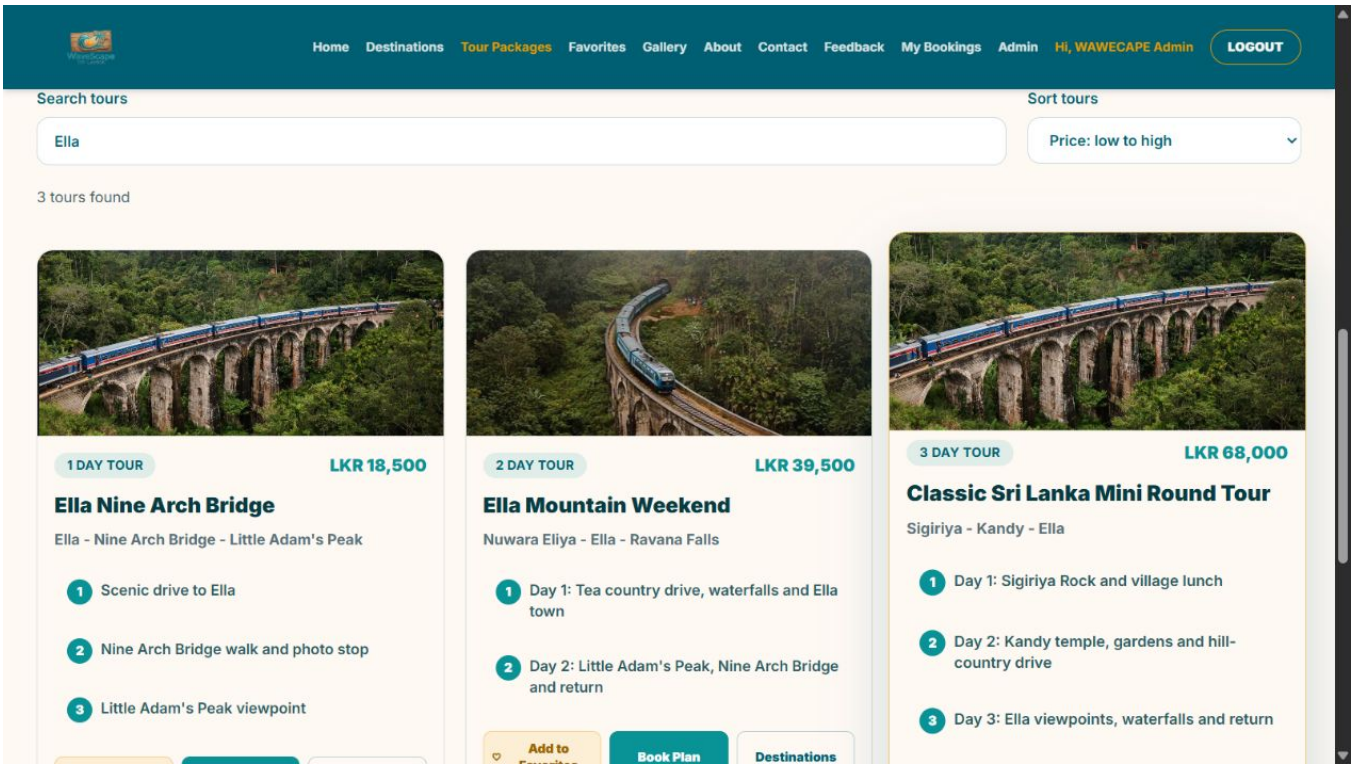


Figure 1. Public database catalogue with search and sorting.

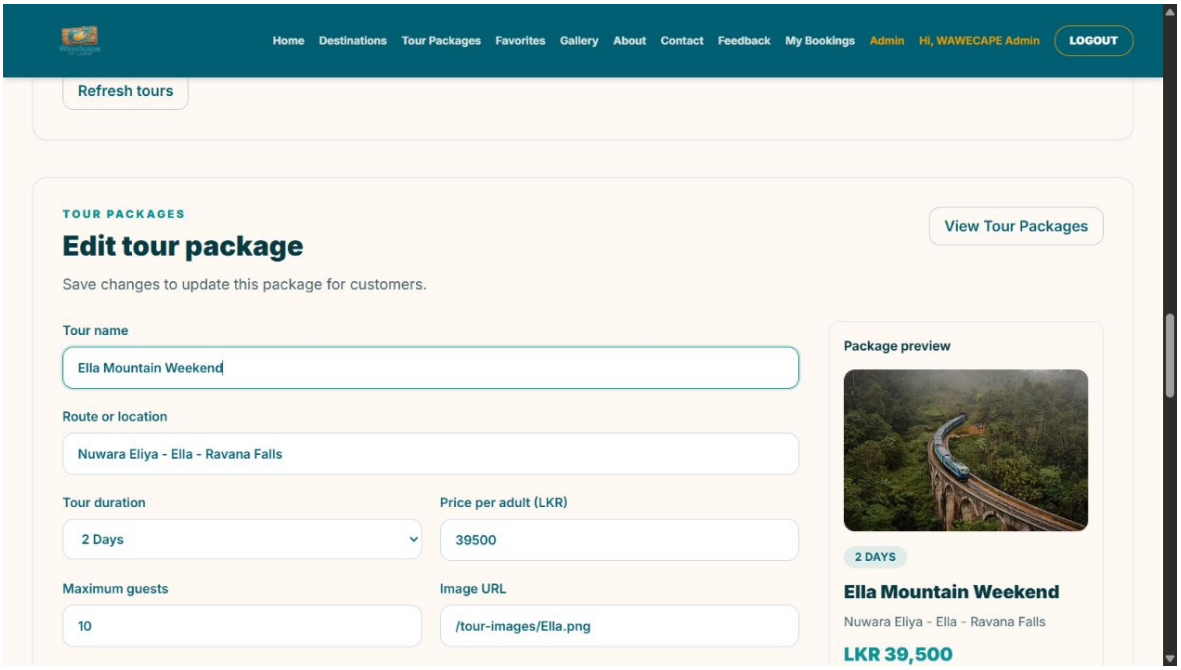


Figure 2. Shared tour form with package preview; creation and editing use the same validated fields.

4. Update and safe Delete

Administrators select Edit to load a tour into the form. Saving updates the same ID; subsequent public reads use the updated record. Visibility can be disabled to prevent new bookings without removing history. All original starter packages are imported into MySQL once and can be managed through this workflow.

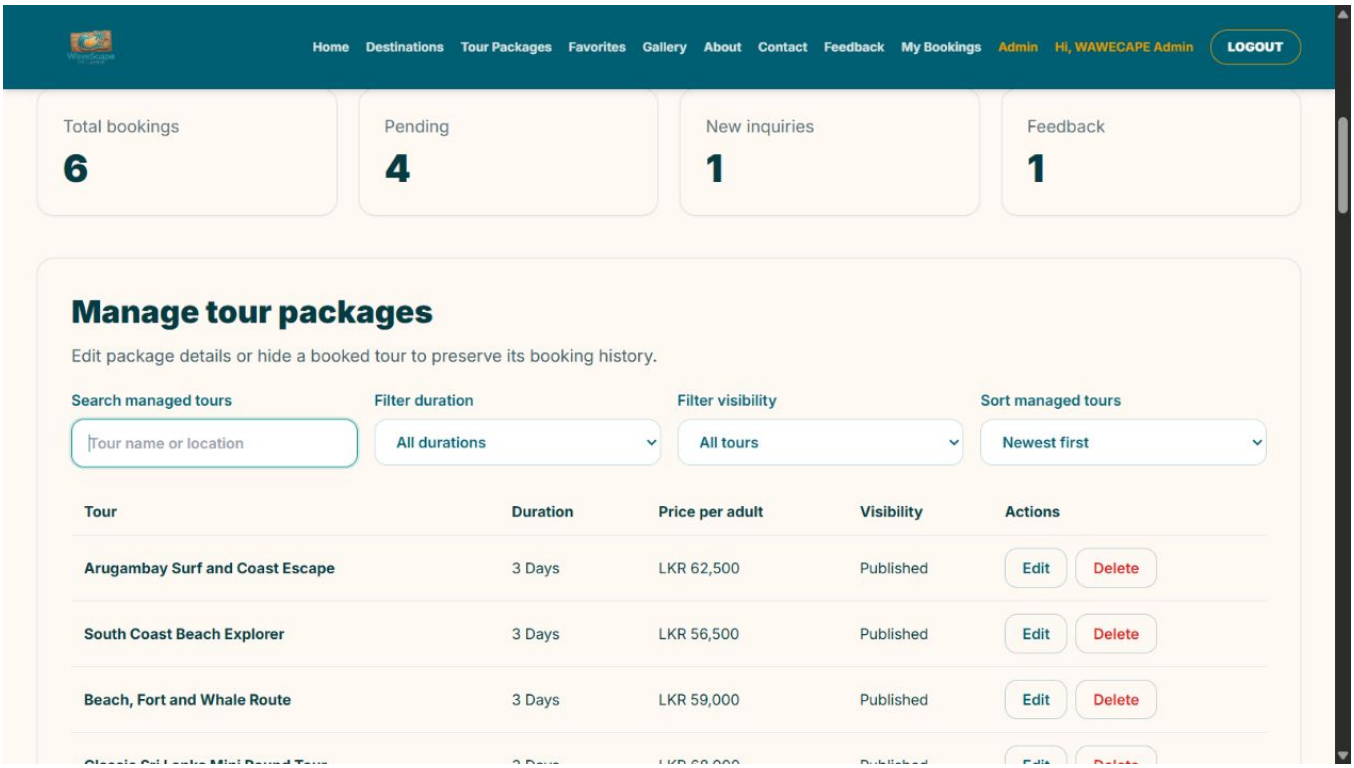


Figure 3. Administrative tour listing and management controls.

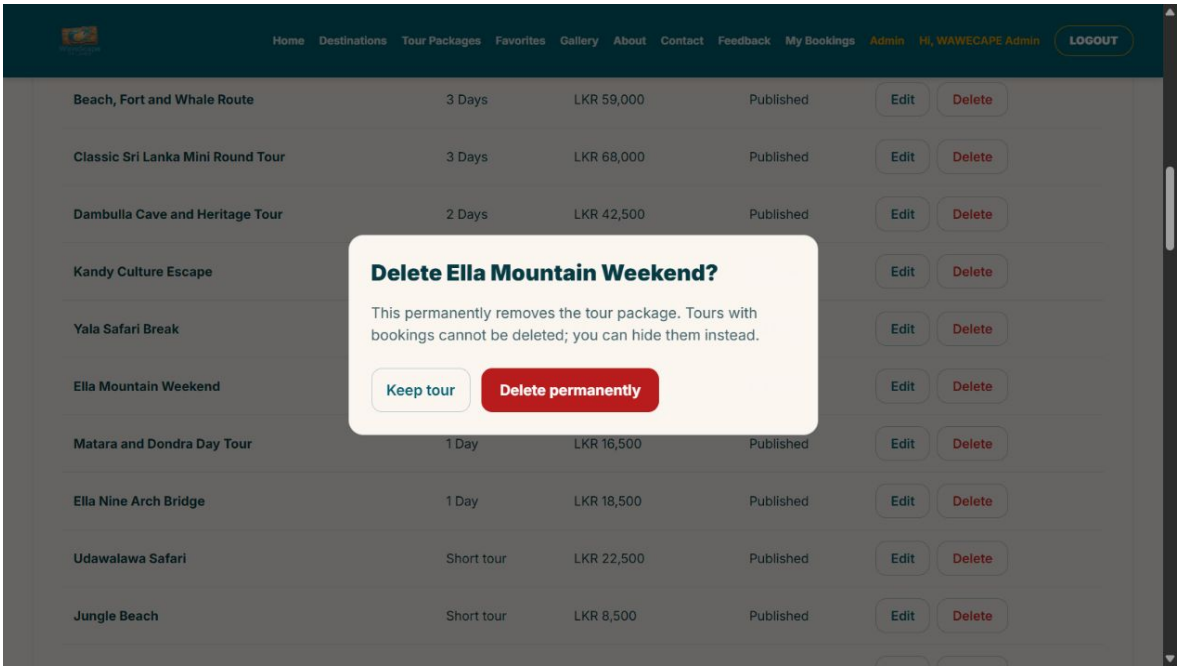


Figure 4. Explicit confirmation before permanent deletion. API checks and foreign keys reject deletion of tours referenced by bookings.

5. Validation, errors and responsive UI

Input or event	Handling
Contact fields	Required text is trimmed and bounded. Emails and phone numbers are validated by the server.
Travel date	Real calendar date, today or later in Sri Lanka. Past or impossible dates are rejected.
Guests and prices	Whole nonnegative guest counts, at least one adult, total capacity check and a positive stored price. Client-supplied booking prices are ignored.
Statuses and roles	Only allowed status values are accepted. Management routes require an authenticated admin.
Request failures	Sanitized JSON errors, visible messages and retry controls. Expired sessions are cleared; invalid local session data cannot crash startup.
Mobile and keyboard use	Collapsed navigation menu, sticky header in document flow, readable contrast, keyboard focus indicators and contained deletion-dialog focus.

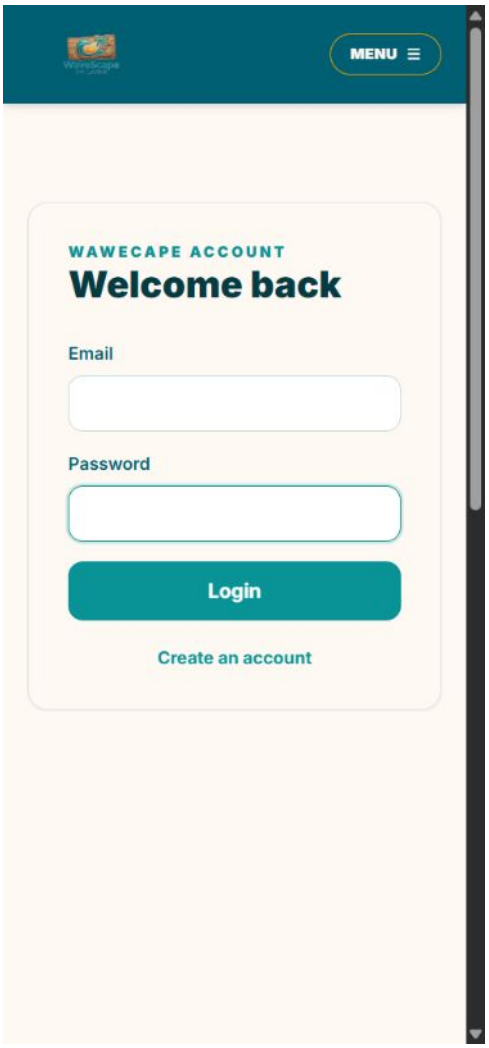


Figure 5. Mobile login page at 390px width; navigation no longer overlaps the heading.

6. Setup and verification

Install Node.js 20 or later and MySQL, create the wavescape_db database, then copy backend/.env.example to backend/.env and set database credentials and a JWT secret. Run npm ci, npm run seed and npm run dev in backend. Run npm ci and npm run dev in frontend. The normal ports are 5001 for the API and 5173 for the client.

Startup creates required tables, imports starter packages once and upgrades legacy booking structures. Re-running seed does not clear bookings or overwrite existing tour edits. Bundled catalogue images are served by the API's /tour-images route.

Check	Evidence
Automated HTTP checks	Role access, publishing, public reads, editing, booked-tour protection, trusted booking price/capacity and sanitized asynchronous failures.
Validation/query checks	Dates, contacts, guest counts and status constraints; search wildcard escaping and sort-expression allowlisting.
Isolated MySQL check	Fresh setup and repeatable legacy migration, including a trip whose numeric ID conflicts with a different current tour.
Frontend checks/build	Package conversion and price formatting tests; Vite production build succeeds.
Browser verification	Temporary tour publication and persistence after refresh, booking-page details, edit loading, delete cancellation, search/price sorting, empty results and mobile layout.

The migration retained the six existing package bookings. Browser screenshots deliberately show catalogue and management controls rather than customer contact records.

Commands and detailed acceptance steps are in docs/TESTING.md. The optional MySQL test creates and removes a generated temporary database; ordinary HTTP tests use an isolated query stub and disable notifications.

Live SMTP/Twilio delivery and the student's camera-on demonstration recording are outside the completed verification.

7. Evaluation coverage and demonstration

Criterion	Marks	Application evidence
Create	4	Admin tour form persists records in MySQL.
Read	4	Catalogue, details, search, duration/visibility filters and sorting.
Update	4	Admin edits the same tour record.
Delete	4	Confirmation and protection of booked tours.
Validation/errors	2	Client constraints, server validation and visible failures.
Database design	4	Keys, relationships, suitable types and compatibility migrations.
UI/UX/mobile	2	Consistent navigation, responsive menu and readable forms.
Code quality	2	Reusable components/utilities, route separation and Git history.
Documentation	2	README, setup, ERD, screenshots, testing guide and this report.
Demonstration	2	Timed guide prepared; student must record and present.

The rubric's full CRUD requirement is addressed through tours as one persistent main entity. This table maps evidence to the assessment categories; it is not a promise of marks.

The accompanying demonstration guide targets 4 minutes 55 seconds: introduce the application, show the ERD, search and sort tours, create a demonstration package, refresh it, edit it, demonstrate safe deletion, and explain validation and tests.

Before submission, review this report, record the camera-on video, confirm the recording is no longer than five minutes and submit the repository link through the lecturer's designated process. The supplied evaluation sheet states a deadline of 31 October 2026.

Report author: MBM Yoonus | EG/2023/5990

Supporting files: README.md; docs/ERD.md; docs/TESTING.md; docs/DEMONSTRATION_GUIDE.md; backend/src/db.js; backend/src/routes/; backend/test/; frontend/src/; frontend/test/.